

RELATÓRIO TÉCNICO: AGRUPAMENTO DE ESPAÇAMENTO MÁXIMO

Técnicas de Busca e Ordenação - 2026/1

Introdução

Este documento descreve a implementação e análise de um algoritmo de agrupamento (clustering) de espaçamento máximo baseado em Árvore Geradoras Mínimas (MST). O objetivo principal foi processar um conjunto de pontos multidimensionais e dividi-los em k grupos distintos, maximizando a distância entre os grupos.

Os resultados obtidos demonstram que a aplicação do algoritmo de Kruskal, modificada para interromper a união de componentes ao atingir o número desejado de grupos, é altamente eficaz. O programa foi validado com sucesso através de testes com arquivos de entrada reais e verificado com o *Valgrind*, apresentando ausência total de vazamentos de memória e erros de execução.

2. Metodologia

A implementação foi dividida em módulos para garantir a extensibilidade e organização do código. As principais decisões de projeto incluem:

- Estrutura Union-Find:** Utilizada para gerir os conjuntos disjuntos durante o algoritmo de Kruskal. Implementamos a técnica de **Path Halving** no *Find* e **Union by Size**, garantindo uma complexidade quase linear.
- Estrutura de Árvore N-ária:** Para facilitar a identificação dos grupos finais e a escrita formatada, cada nó do Union-Find aponta para uma estrutura de árvore física. Quando duas componentes são unidas, as suas árvores são conectadas fisicamente (um nó torna-se filho/irmão do outro).
- Algoritmo de Kruskal:** Escolhido pela sua naturalidade em lidar com problemas de espaçamento máximo. Ao ordenar as arestas e unir os pontos mais próximos primeiro, as últimas arestas que "não foram colocadas" representam justamente o espaçamento máximo entre os clusters.
- Ordenação:** O uso do `qsort` da biblioteca padrão foi aplicado tanto para as distâncias (arestas) quanto para a formatação da saída (nomes dos pontos), garantindo eficiência $O(n \log n)$.

3. Análise de Complexidade

Considerando n como o número de pontos e m como o número de arestas ($m = n(n-1)/2$):

- **Cálculo das Distâncias:** $O(n^2)$, pois calculamos a distância entre cada par de pontos.
- **Ordenação das Arestas:** $O(m \log m)$. Como $m \approx n^2$, isto equivale a $O(n^2 \log n)$.
- **Algoritmo de Kruskal (Union-Find):** $O(m \alpha(n))$, onde α é a função inversa de Ackermann (praticamente constante).
- **Identificação dos Grupos:** $O(n \log n)$ devido à necessidade de ordenar os nomes dos integrantes de cada grupo antes da impressão.

Complexidade Total: $O(n^2 \log n)$, dominada pela ordenação de todas as distâncias possíveis entre os pontos.

4. Análise Empírica

Abaixo, apresentamos os dados coletados na execução do caso de teste 5.txt (5000 pontos) para $k=10$.

Etapa	Tempo Gasto (%)
Leitura dos dados	0,004s - 0,12%
Cálculo das distâncias	0,793s - 19,19%
Ordenação das distâncias	3,324s - 80,43%
Obtenção da MST (Kruskal)	0,006s - 0,15%
Identificação dos grupos e escrita	0,002s - 4%
TOTAL	4,133s – 100%

Conclusão: Os valores obtidos estão em estreita concordância com a análise de complexidade. As etapas de **Cálculo** e **Ordenação** consomem a maior parte do tempo (99,62% no total), o que é esperado, dado que ambas operam sobre a ordem de n^2 elementos. A eficiência do Union-Find reflete-se no baixo tempo gasto na obtenção da MST, mesmo sendo o núcleo lógico do programa.